



ROADMAP DE DÉVELOPPEMENT

Zümm

Planification Scrum & pipeline DevOps

Projet	Application de gestion apicole (mini-projet J2EE)
Type	Roadmap de développement (Scrum + DevOps)
Version	1.0
Date	16 juillet 2026
Référence	Cahier des charges Zümm v1.0 (13 juillet 2026)
Cadence	8 sprints de 14 jours — 14/07/2026 → 02/11/2026

10 epics — 40 user stories — 304 story points

Document conforme au plan : Méthodologie / Product Backlog / Sprints /
Pipeline DevOps / Plan de releases / Monitoring & observabilité

Table des matières

Historique des versions	2
1 Méthodologie : Scrum + DevOps	3
1.1 Objet du document	3
1.2 Cadre Scrum	3
1.2.1 Rôles	3
1.2.2 Cérémonies	3
1.2.3 Definition of Ready (DoR)	4
1.2.4 Definition of Done (DoD)	4
1.3 Principes DevOps	4
2 Product Backlog	5
2.1 Vue d'ensemble des epics	5
2.2 EPIC-001 — Gestion des entités métier (CRUD)	5
2.3 EPIC-002 — Planification et suivi des visites	6
2.4 EPIC-003 — Tableaux de bord et visualisation	6
2.5 EPIC-004 — Indicateurs et capteurs (préparation)	7
2.6 EPIC-005 — Authentification et sécurité	7
2.7 EPIC-006 — Internationalisation et configuration	7
2.8 EPIC-007 — Service web API tierce	7
2.9 EPIC-008 — Fonctionnalités avancées (inspiration marché)	8
2.10 EPIC-009 — Intelligence artificielle (préparation)	8
2.11 EPIC-010 — Tests, qualité et documentation	8
3 Planification des sprints	9
3.1 Calendrier général	9
3.2 Synthèse des sprints	10
3.3 Recommandations d'équilibrage et de pilotage	10
3.3.1 Options d'équilibrage de la charge	10
3.3.2 Ordre des dépendances	10
3.3.3 Pilotage	11
3.3.4 Recommandations techniques transversales	11
3.4 Sprint 1 — Fondation : CRUD Core	11
3.4.1 Plan d'exécution	12
3.5 Sprint 2 — Ruche & composition	12

3.5.1	Plan d'exécution	13
3.6	Sprint 3 — Visites & rapports	13
3.6.1	Plan d'exécution	14
3.7	Sprint 4 — Hors-ligne & synchronisation	14
3.7.1	Plan d'exécution	15
3.8	Sprint 5 — Tableaux de bord	15
3.8.1	Plan d'exécution	16
3.9	Sprint 6 — Capteurs & API	16
3.9.1	Plan d'exécution	17
3.10	Sprint 7 — Fonctions avancées	17
3.10.1	Plan d'exécution	18
3.11	Sprint 8 — Qualité & livraison	18
3.11.1	Plan d'exécution	19
3.12	Suivi en cours de sprint	19
4	Pipeline DevOps (CI/CD)	20
4.1	Environnements	20
4.2	Vue d'ensemble du pipeline	20
4.3	Détail des étapes	21
4.4	Infrastructure as Code	21
4.5	Mise en place progressive	21
4.6	Stratégie de branches Git	22
5	Plan de releases	23
5.1	Synthèse	23
5.2	v0.1.0 — MVP : Fondation	23
5.3	v0.2.0 — Visites & collaboration	23
5.4	v0.3.0 — Analytics & dashboards	24
5.5	v1.0.0 — Release Candidate	24
6	Monitoring & observabilité	25
6.1	Dashboards Grafana	25
6.1.1	Zümm — Vue d'ensemble (métier)	25
6.1.2	Zümm — Performance (technique)	25
6.1.3	Zümm — Sécurité	25
6.2	Règles d'alerte	26
6.3	Gestion des logs	26

Historique des versions

Version	Date	Auteur	Modifications
1.0	16 juillet 2026	Équipe projet	Consolidation LaTeX du dossier <code>zumm_scrum_devops_roadmap</code> : backlog, sprints, pipeline CI/CD, releases et monitoring. Correction des totaux de points des sprints 5, 7 et 8.

CHAPITRE 1

Méthodologie : Scrum + DevOps

1.1 Objet du document

Ce document constitue la **roadmap de développement** du projet Zümm. Il traduit le cahier des charges (v1.0, 13 juillet 2026) en un plan d'exécution opérationnel : un *product backlog* priorisé, une planification en **8 sprints** de deux semaines, une chaîne d'intégration et de déploiement continu (CI/CD), un plan de releases sémantiques et un dispositif de monitoring.

Chiffres clés. 10 epics — 40 user stories — 304 story points — 8 sprints de 14 jours — 4 releases (v0.1.0 → v1.0.0) — période : 14 juillet → 2 novembre 2026.

1.2 Cadre Scrum

1.2.1 Rôles

Rôle	Responsabilité
Product Owner	Priorise le backlog, valide les critères d'acceptation, accepte les livrables en sprint review.
Scrum Master	Garant du cadre, lève les obstacles, anime les cérémonies.
Équipe de développement	Conçoit, développe, teste et livre l'incrément de chaque sprint.

1.2.2 Cérémonies

Chaque sprint suit le même rythme de cérémonies :

Cérémonie	Cadence / durée	Objectif
Sprint Planning	Jour 1, 09h00–13h00 (4 h)	Sélection des stories et engagement de l'équipe.
Daily Scrum	Tous les jours, 09h15 (15 min)	Synchronisation, détection des blocages.
Sprint Review	Dernier jour, 14h00–16h00 (2 h)	Démonstration de l'incrément au Product Owner.
Sprint Retrospective	Dernier jour, 16h00–17h00 (1 h)	Amélioration continue du processus.

1.2.3 Definition of Ready (DoR)

Une story ne peut entrer dans un sprint que si :

1. elle a un titre clair et une description ;
2. ses critères d'acceptation sont définis ;
3. ses dépendances sont identifiées ;
4. elle est estimée en points ;
5. le Product Owner a validé sa priorité.

1.2.4 Definition of Done (DoD)

Une story n'est terminée que si :

1. le code est développé et revu (*peer review*) ;
2. les tests unitaires passent (couverture $\geq 70\%$) ;
3. les tests d'intégration passent ;
4. la documentation est mise à jour ;
5. l'incrément est déployé sur l'environnement de staging ;
6. le Product Owner a validé la story.

1.3 Principes DevOps

La démarche DevOps complète Scrum sur l'axe technique :

- **Intégration continue** : chaque *push* déclenche build, tests et analyse qualité (chapitre 4) ;
- **Déploiement continu** : livraison automatique en staging, déploiement en production sur validation manuelle (*approval gate*) ;
- **Infrastructure as Code** : Docker, docker-compose, manifestes Kubernetes optionnels ;
- **Observabilité** : logs centralisés, métriques Prometheus, dashboards Grafana, alerting (chapitre 6) ;
- **Boucle de rétroaction** : les métriques de production alimentent la priorisation du backlog.

Note de cohérence. Les fichiers source `sprints.json` et `releases.json` annonçaient 284 points planifiés ; la somme réelle des stories affectées aux sprints est de **304 points** (écarts sur les sprints 5, 7 et 8). Ce document retient les valeurs recalculées, cohérentes avec le product backlog.

CHAPITRE 2

Product Backlog

Le backlog est structuré en **10 epics** couvrant l'intégralité du périmètre du cahier des charges, pour un total de **40 user stories** et **304 story points** (suite de Fibonacci : 1, 2, 3, 5, 8, 13).

2.1 Vue d'ensemble des epics

Epic	Intitulé	Priorité	Source CdC	Points
EPIC-001	Gestion des entités métier (CRUD)	Haute	§4.1, §5.2, §7.1	44
EPIC-002	Planification et suivi des visites	Haute	§4.2, §6.1, §6.2	44
EPIC-003	Tableaux de bord et visualisation	Haute	§4.3	42
EPIC-004	Indicateurs et capteurs (préparation)	Moyenne	§4.4	29
EPIC-005	Authentification et sécurité	Haute	§8.2, §8.3, Ann. I	26
EPIC-006	Internationalisation et configuration	Haute	§8.2, §8.3	13
EPIC-007	Service web API tierce	Moyenne	§6.5, §8.2	10
EPIC-008	Fonctionnalités avancées (marché)	Moyenne	§4.6, §4.7	36
EPIC-009	Intelligence artificielle (préparation)	Moyenne	Annexe H	21
EPIC-010	Tests, qualité et documentation	Haute	Chap. 10, 11, Ann. F	39
Total				304

2.2 EPIC-001 — Gestion des entités métier (CRUD)

Opérations CRUD sur fermiers, fermes, sites, ruches, corps/hausses, cadres et agents.

ID	Story	Pts	Priorité	Critères d'acceptation
US-001	CRUD Fermier	5	Haute	CRUD complet avec validation
US-002	CRUD Ferme	5	Haute	Liaison fermier-ferme
US-003	CRUD Site (avec géolocalisation)	8	Haute	Lat/long/altitude + Post-GIS
US-004	CRUD Ruche avec composition	13	Haute	1 corps + 0-5 hausses + 1-10 cadres
US-005	CRUD Agent avec rôles	5	Haute	Rôles : apiculteur, superviseur, responsable, admin
US-006	Contraintes de composition (métier)	8	Haute	Contraintes CHECK SQL + validation Java

2.3 EPIC-002 — Planification et suivi des visites

Planification, approbation, réalisation et rapport de visite.

ID	Story	Pts	Priorité	Critères d'acceptation
US-007	Planifier une visite	8	Haute	Date, raison, actions prévues
US-008	Approuver/refuser un planning	5	Haute	Workflow d'approbation superviseur
US-009	Réaliser une visite et remplir le rapport	13	Haute	Date, heure, durée, constatations, évaluations 1-3
US-010	Ajouter des photos au rapport	5	Haute	Upload multi-photos
US-011	Mode hors-ligne et synchronisation différée	13	Haute	PWA + Service Worker + IndexedDB

2.4 EPIC-003 — Tableaux de bord et visualisation

Calendrier, production, alertes, synthèse ROI.

ID	Story	Pts	Priorité	Critères d'acceptation
US-012	Calendrier matrice agents × ruches	13	Haute	Filtres mois/semaine/saison/année + pop-up
US-013	Tableau de bord production	8	Haute	Seuils paramétrables + collecte/extension
US-014	Tableau de bord alertes sanitaires	8	Haute	Baisse anormale détectée + inspection
US-015	Tableau de bord synthèse et ROI	13	Moyenne	Graphiques production, interventions, ROI

2.5 EPIC-004 — Indicateurs et capteurs (préparation)

Modèle de données ouvert pour l'intégration future des capteurs.

ID	Story	Pts	Priorité	Critères d'acceptation
US-016	Modèle de données Mesure	8	Moyenne	Horodatage, géolocalisation, unités paramétrables
US-017	Interface ingestion REST/MQTT	8	Moyenne	POST <code>/api/mesures</code> + topic MQTT
US-018	Seuils paramétrables et logique d'alerte	8	Moyenne	Anti-rebond / hystérésis
US-019	Conversion d'unités hétérogènes	5	Moyenne	Patron <i>Strategy</i>

2.6 EPIC-005 — Authentification et sécurité

OAuth Google, RBAC, SSL/X.509.

ID	Story	Pts	Priorité	Critères d'acceptation
US-020	Authentification OAuth 2.0 Google	8	Haute	Flot <i>Authorization Code</i> + JWT
US-021	Authentification locale (repli)	5	Haute	Repli si OAuth indisponible
US-022	Contrôle d'accès RBAC	8	Haute	Matrice 6 profils × 15 fonctions
US-023	Chiffrement SSL/TLS X.509	5	Haute	HTTPS forcé + certificats

2.7 EPIC-006 — Internationalisation et configuration

i18n FR/EN/AR, `ConfigZumm.ini`.

ID	Story	Pts	Priorité	Critères d'acceptation
US-024	Internationalisation (FR/EN/AR)	XML 8	Haute	RTL arabe + extensible
US-025	Configuration <code>ConfigZumm.ini</code>	5	Haute	Seuils, unités, modules sans recompilation

2.8 EPIC-007 — Service web API tierce

Exposition d'une API pour applications externes.

ID	Story	Pts	Priorité	Critères d'acceptation
US-026	Service web <code>getZummHoneyActualQuantity</code>	XML 5	Moyenne	JAX-WS SOAP + WSDL
US-027	Export CSV/TXT	5	Moyenne	Données maîtrisées par l'utilisateur

2.9 EPIC-008 — Fonctionnalités avancées (inspiration marché)

Fonctions complémentaires inspirées de HiveTracks, Apiary Book et BeePlus.

ID	Story	Pts	Priorité	Critères d'acceptation
US-028	Photos d'inspection	5	Haute	Attachées au rapport
US-029	Contexte météo local	5	Moyenne	API météo par géolocalisation
US-030	Carte et rayons de butinage	8	Moyenne	Leaflet + OpenStreetMap + cercles 1/2/3 km
US-031	Liste de tâches et rappels	5	Haute	Calendrier de rappels
US-032	Suivi de la reine	5	Haute	Historique du statut de la reine
US-033	Récolte et QR code	8	Moyenne	QR par lot + traçabilité

2.10 EPIC-009 — Intelligence artificielle (préparation)

Détection d'anomalie adaptative, pipeline IA.

ID	Story	Pts	Priorité	Critères d'acceptation
US-034	Détection d'anomalie adaptative (EWMA)	13	Moyenne	Ligne de base par ruche + z-score
US-035	Interface microservice IA Python	8	Moyenne	REST/JSON découplé

2.11 EPIC-010 — Tests, qualité et documentation

Plan de tests, UML, rapport, poster.

ID	Story	Pts	Priorité	Critères d'acceptation
US-036	Tests unitaires JUnit 5 + Mockito	8	Haute	70 % de couverture métier
US-037	Tests d'intégration (Arquillian)	5	Haute	Tests <i>in-container</i>
US-038	Tests de charge k6	5	Moyenne	p95 < 500 ms, erreurs < 1 %
US-039	Diagrammes UML complets	13	Haute	10 diagrammes requis
US-040	Rapport + poster + présentation	8	Haute	Livrables de soutenance

CHAPITRE 3

Planification des sprints

Le développement est découpé en **8 sprints de 14 jours** (16 semaines), du 14 juillet au 2 novembre 2026, avec une capacité de référence de 38 points par sprint. Chaque sprint est décrit ci-dessous avec son *plan d'exécution* : décomposition en tâches techniques, estimation en jours-homme (j-h, base 10 jours ouvrés par sprint), livrables et parades aux risques.

3.1 Calendrier général



Figure 3.1. Diagramme de Gantt des 8 sprints et des 4 jalons de release.

3.2 Synthèse des sprints

#	Période	Thème	Points ¹	Stories
1	14/07 → 27/07	Fondation — CRUD Core	31	US-001/002/003/005/006
2	28/07 → 10/08	Ruche & composition	26	US-004/024/025
3	11/08 → 24/08	Visites & rapports	31	US-007/008/009/010
4	25/08 → 07/09	Hors-ligne & synchronisation	26	US-011/020/021
5	08/09 → 21/09	Tableaux de bord	50	US-012 à 015, US-022
6	22/09 → 05/10	Capteurs & API	39	US-016 à 019, US-026/027
7	06/10 → 19/10	Fonctions avancées	57	US-028 à 035
8	20/10 → 02/11	Qualité & livraison	44	US-023, US-036 à 040
Total			304	40 stories

Alerte de charge. Les sprints 5 (50 pts) et 7 (57 pts) excèdent la vélocité cible de 38 points, tandis que les sprints 2 et 4 sont sous-chargés (26 pts). Les options d'équilibrage sont analysées en section 3.3.

3.3 Recommandations d'équilibrage et de pilotage

3.3.1 Options d'équilibrage de la charge

Option	Principe	Effet
A. Descoper (MoSCoW)	Marquer US-015 (ROI, 13 pts) et US-034 (EWMA, 13 pts) <i>Could have</i> : livrées seulement si la vélocité réelle le permet.	S5 → 37 pts, S7 → 44 pts, calendrier inchangé.
B. Lisser sur les sprints creux	S2 et S4 sont à 26 pts : avancer US-028 (photos, dépend du rapport → S3/S4) et US-031/US-032 (rappels, reine — 10 pts) vers S4.	S7 → 42 pts, S4 → 36 pts.
C. Scinder les US lourdes	US-012 (calendrier, 13 pts) → « affichage matrice » (8) + « filtres/pop-up » (5) ; idem US-034.	Risque d'échec de sprint réduit, burndown plus lisse.
D. Buffer explicite	Ne planifier que $\approx 90\%$ de la capacité (34 pts) et réserver le reste aux bugs et à la dette technique.	Planification plus réaliste qu'une capacité pleine.

3.3.2 Ordre des dépendances

L'ordre de réalisation compte davantage que la charge :

- **US-022 (RBAC) est trop tardive en S5** : le workflow d'approbation superviseur (US-008, S3) suppose déjà des rôles. Recommandation : avancer le RBAC en S3, quitte à repousser US-010 (photos) ;

1. Points recalculés depuis le backlog. Les sprints 5 et 7 dépassent la capacité de référence (38 pts) : un rééquilibrage vers les sprints 2 et 4 (26 pts) est recommandé lors des sprint plannings.

- **US-024 (i18n RTL arabe) en S2** : bien placée, mais le rendu RTL doit être testé dès la première maquette — le rétrofit RTL est le piège classique ;
- **Spike technique en début de S1** (2–3 jours) sur PostGIS + WildFly : c’est le risque identifié du sprint, autant le neutraliser tôt. De même, prototyper Service Worker/IndexedDB pendant S3 pour dérisquer S4.

3.3.3 Pilotage

- **Recalibrer après S2** : la vitesse de 38 pts est une hypothèse ; après deux sprints mesurés, replanifier S5–S8 avec la vitesse réelle (c’est là que le choix entre les options A et B se décide) ;
- **Release interne à chaque sprint** (tag `v0.x.y-sprintN`) même si seules les 4 releases publiques comptent : cela teste le pipeline de release avant la `v1.0.0` ;
- **Calendrier** : si le projet ne démarre pas réellement le 14/07, décaler toutes les dates de deux semaines est préférable à un sprint 1 amputé.

3.3.4 Recommandations techniques transversales

- **Migrations de schéma dès S1** (Flyway ou Liquibase) : chaque évolution du modèle est un script versionné — indispensable avec 4 environnements et des contraintes CHECK qui évoluent ;
- **Décisions d’architecture tracées** (ADR — *Architecture Decision Records*) : un fichier court par choix structurant (JSF vs REST+SPA, stockage des photos, stratégie de synchronisation) — réutilisable tel quel dans le rapport de maintenance ;
- **Stockage des photos** : système de fichiers (ou S3/MinIO) + chemin en base, jamais de BLOB en base — avec limite de taille et génération de miniatures dès l’upload ;
- **Revue de code systématique** : chaque PR relue par un pair avant merge sur `develop` (exigence déjà présente dans la DoD, à outiller par une *branch protection rule*) ;
- **Registre des risques** tenu à jour à chaque rétrospective : les risques par sprint listés ci-dessous en sont l’état initial ;
- **UML au fil de l’eau** : commencer les diagrammes dès S2 et les maintenir à chaque sprint plutôt que de tout produire en S8 (US-039, 13 pts, est la plus grosse story du sprint final) ;
- **Gel des fonctionnalités** (*feature freeze*) au jour 10 du sprint 8 : les 4 derniers jours sont réservés aux corrections, aux répétitions de maintenance et à l’empaquetage des livrables.

3.4 Sprint 1 — Fondation : CRUD Core

Objectif	Avoir un socle technique fonctionnel avec les entités principales.
Période	14/07/2026 → 27/07/2026 (14 jours) — 31 pts / capacité 38.
Démo	Démonstration CRUD Fermier/Ferme/Site/Agent.
Risques	Complexité PostGIS, configuration WildFly.
Burndown idéal	J1 : 31 — J4 : 23 — J7 : 15 — J10 : 8 — J14 : 0.

ID	Story	Pts
US-001	CRUD Fermier	5
US-002	CRUD Ferme	5
US-003	CRUD Site (avec géolocalisation)	8
US-005	CRUD Agent avec rôles	5
US-006	Contraintes de composition (règles métier)	8

3.4.1 Plan d'exécution

Tâche	Stories	j-h	Livrable
Spike PostGIS + WildFly : datasource, dialecte spatial, requête lat/long de bout en bout	US-003	2	Note technique + config <code>standalone.xml</code> versionnée
Squelette Maven multi-modules (ejb/web/ear) + CI GitHub Actions (build + tests)	—	1,5	Dépôt buildable, pipeline vert
Modèle JPA (Fermier, Ferme, Site, Agent) + migrations Flyway	US-001/002/003/005	2	Schéma V1 reproductible
Couche DAO/Service + Bean Validation	US-001/002/003/005	2	Services testables
Écrans CRUD des 4 entités	US-001/002/003/005	3	4 CRUD démontrables
Contraintes de composition : CHECK SQL + validateurs Java	US-006	2	Règles métier vérifiées aux deux niveaux
Tests unitaires des services + jeu de données de démonstration	toutes	1,5	Couverture initiale $\geq 50\%$, seed SQL

Parades aux risques : le spike PostGIS occupe les jours 1–2 ; si l'intégration spatiale bloque, solution de repli : colonnes `lat/long` simples en `NUMERIC` et migration PostGIS différée au sprint 5 (les dashboards n'en dépendent pas avant).

3.5 Sprint 2 — Ruche & composition

Objectif	Modéliser la hiérarchie ruche/corps/hausse/cadre avec contraintes.
Période	28/07/2026 → 10/08/2026 (14 jours) — 26 pts / capacité 38.
Démo	Démonstration de la composition d'une ruche avec règles métier.
Risques	Contraintes SQL CHECK, patron <i>Composite</i> .
Burndown idéal	J1 : 26 — J4 : 20 — J7 : 13 — J10 : 7 — J14 : 0.

ID	Story	Pts
US-004	CRUD Ruche avec composition	13
US-024	Internationalisation XML (FR/EN/AR)	8
US-025	Configuration <code>ConfigZumm.ini</code>	5

3.5.1 Plan d'exécution

Tâche	Stories	j-h	Livrable
Modèle <i>Composite</i> ruche/corps/hausse/cadre + contraintes (1 corps, 0-5 hausses, 1-10 cadres/élément)	US-004	3	Migration V2 + entités JPA
Interface de composition (vue arborescente, ajout/retrait d'éléments avec validation immédiate)	US-004	2,5	Écran composition démontable
Lecteur <code>ConfigZumm.ini</code> + rechargement sans redéploiement	US-025	1,5	Module de configuration
i18n XML FR/EN/AR : externalisation des libellés, bascule de langue, gabarit RTL	US-024	2,5	Application trilingue
Test de rendu RTL sur les écrans existants (S1 + composition)	US-024	0,5	Rapport d'écarts RTL
Socle de tests d'intégration Arquillian (déploiement in-container + H2)	—	1,5	Premier test d'intégration vert en CI
Déploiement continu staging (docker-compose)	—	1	Environnement staging alimenté à chaque merge

Parades aux risques : implémenter les règles de composition d'abord en Java (testable rapidement), puis répliquer en CHECK SQL ; si le patron *Composite* complique le mapping JPA, se limiter à une hiérarchie à trois tables avec clés étrangères et compteurs contraints.

Marge disponible (26 pts < 38) : absorber ici US-031/US-032 (10 pts) si l'option B d'équilibrage est retenue.

3.6 Sprint 3 — Visites & rapports

Objectif	Workflow complet : planification → approbation → réalisation → rapport.
Période	11/08/2026 → 24/08/2026 (14 jours) — 31 pts / capacité 38.
Démo	Démonstration du workflow de visite avec rapport complet.
Risques	Mode hors-ligne (reporté au sprint 4).
Burndown idéal	J1 : 31 — J4 : 23 — J7 : 15 — J10 : 8 — J14 : 0.

ID	Story	Pts
US-007	Planifier une visite	8
US-008	Approuver/refuser un planning	5
US-009	Réaliser une visite et remplir le rapport	13
US-010	Ajouter des photos au rapport	5

3.6.1 Plan d'exécution

Tâche	Stories	j-h	Livrable
Modèle Visite/Planning/Rapport + machine à états (planifiée → approuvée → réalisée / refusée)	US-007/008/009	2,5	Migration V3 + diagramme d'états
RBAC minimal : rôles portés par Agent + garde d'accès sur le workflow (anticipation d'US-022, cf. §3.3)	US-008	2	Approbation réservée au superviseur
Écrans de planification (date, raison, actions prévues)	US-007	2	Formulaire de planification
File d'approbation du superviseur (accepter/refuser + motif)	US-008	1,5	Écran d'approbation
Formulaire de rapport : date, heure, durée, constatations, évaluations 1-3	US-009	2,5	Rapport complet saisissable
Upload multi-photos : stockage fichiers + miniatures + limite de taille	US-010	2	Photos attachées au rapport
Spike Service Worker + IndexedDB (préparation S4)	—	1	Prototype de mise en cache
Tests unitaires du workflow (transitions interdites)	US-007/008/009	1	Machine à états couverte

Parades aux risques : le report du mode hors-ligne est déjà acté (S4) ; le spike Service Worker de fin de sprint sécurise cette échéance. Verrouiller la machine à états par des tests avant de brancher l'interface.

3.7 Sprint 4 — Hors-ligne & synchronisation

Objectif	Permettre la saisie terrain sans réseau.
Période	25/08/2026 → 07/09/2026 (14 jours) — 26 pts / capacité 38.
Démo	Démonstration PWA hors-ligne + OAuth.
Risques	Service Workers, IndexedDB, synchronisation différée.
Burndown idéal	J1 : 26 — J4 : 20 — J7 : 13 — J10 : 7 — J14 : 0.

ID	Story	Pts
US-011	Mode hors-ligne et synchronisation différée	13
US-020	Authentification OAuth 2.0 Google	8
US-021	Authentification locale (repli)	5

3.7.1 Plan d'exécution

Tâche	Stories	j-h	Livrable
PWA : manifest + Service Worker (cache de l'app shell)	US-011	2	Application installable
File d'attente IndexedDB des saisies hors-ligne (visites, rapports)	US-011	2,5	Saisie sans réseau fonctionnelle
Synchronisation différée : rejeu à la reconnexion, idempotence, résolution de conflits (<i>last-write-wins</i> + journal)	US-011	2,5	Démo « mode avion »
OAuth 2.0 Google : flot <i>Authorization Code</i> + émission JWT	US-020	2	Connexion Google opérationnelle
Authentification locale de repli (hachage BCrypt/Argon2)	US-021	1	Repli fonctionnel hors OAuth
Gestion des secrets dans le pipeline (GitHub Secrets)	US-020	0,5	Aucun secret en clair dans le dépôt
Tests E2E du parcours hors-ligne (Cypress, réseau coupé)	US-011	1,5	Scénario automatisé en CI

Parades aux risques : limiter le périmètre hors-ligne à la *saisie de visite/rapport* (pas de consultation complète) ; en cas de blocage sur la résolution de conflits, dégrader en « premier arrivé, premier servi » avec signalement manuel des collisions.

Marge disponible (26 pts < 38) : absorber ici US-015 (13 pts) ou US-028 selon l'option d'équilibrage retenue.

3.8 Sprint 5 — Tableaux de bord

Objectif	Calendrier, production, alertes, synthèse.
Période	08/09/2026 → 21/09/2026 (14 jours) — 50 pts / capacité 38 (surcharge, cf. §3.3).
Démo	Démonstration des dashboards avec filtres et graphiques.
Risques	Performance des agrégations SQL, Chart.js.
Burndown idéal	J1 : 50 — J4 : 38 — J7 : 25 — J10 : 13 — J14 : 0.

ID	Story	Pts
US-012	Calendrier matrice agents × ruches	13
US-013	Tableau de bord production	8
US-014	Tableau de bord alertes sanitaires	8
US-015	Tableau de bord synthèse et ROI	13
US-022	Contrôle d'accès RBAC	8

3.8.1 Plan d'exécution

Tâche	Stories	j-h	Livrable
Vues SQL agrégées + index (production, visites par période)	US-012/013	2	Requêtes < 200 ms sur données de test
Calendrier matrice agents × ruches : affichage + pop-up de détail	US-012	2	Matrice navigable
Filtres mois/semaine/saison/année du calendrier	US-012	1	Filtres opérationnels
Dashboard production (Chart.js) + seuils paramétrables via <code>ConfigZumm.ini</code>	US-013	2	Graphiques de production
Dashboard alertes sanitaires : règle de baisse anormale (seuil simple, l'EWMA arrive en S7)	US-014	2	Liste d'alertes + statut inspection
Généralisation RBAC : matrice 6 profils × 15 fonctions (extension du RBAC minimal de S3)	US-022	1,5	Matrice appliquée à tous les écrans
(<i>Could have</i>) Synthèse ROI : production, interventions, coûts	US-015	2	Page de synthèse
Test de charge léger des dashboards (k6, requêtes agrégées)	—	0,5	Rapport p95

Parades aux risques : matérialiser les agrégations coûteuses (vues matérialisées rafraîchies à la demande) plutôt que d'optimiser des requêtes à chaud ; US-015 est le fusible du sprint — descopée en premier si le burndown dérive (option A).

3.9 Sprint 6 — Capteurs & API

Objectif	Architecture ouverte pour capteurs et API tierce.
Période	22/09/2026 → 05/10/2026 (14 jours) — 39 pts / capacité 38.
Démo	Démonstration de l'ingestion de mesures + API SOAP.
Risques	MQTT, conversion d'unités, idempotence.
Burndown idéal	J1 : 39 — J4 : 29 — J7 : 20 — J10 : 10 — J14 : 0.

ID	Story	Pts
US-016	Modèle de données Mesure	8
US-017	Interface ingestion REST/MQTT	8
US-018	Seuils paramétrables et logique d'alerte	8
US-019	Conversion d'unités hétérogènes	5
US-026	Service web XML <code>getZummHoneyActualQuantity</code>	5
US-027	Export CSV/TXT	5

3.9.1 Plan d'exécution

Tâche	Stories	j-h	Livrable
Entité Mesure (horodatage, géolocalisation, unité, source) + migration	US-016	1,5	Modèle extensible capteurs
API REST d'ingestion POST /api/mesures idempotente (clé d'unicité source + horodatage)	US-017	2	Endpoint documenté (OpenAPI)
Broker MQTT (Mosquitto en docker-compose) + consommateur	US-017	2	Ingestion par topic démontrable
Seuils paramétrables + hystérésis/anti-rebond	US-018	1,5	Alertes sans battement
Conversion d'unités (patron <i>Strategy</i>)	US-019	1	Conversions couvertes par tests
Service SOAP JAX-WS getZummHoneyActualQuantity + WSDL	US-026	1,5	WSDL consommable par client tiers
Export CSV/TXT des données maîtrisées par l'utilisateur	US-027	1	Export téléchargeable
Tests de contrat des API (REST + SOAP)	US-017/026	1	Suites de contrat en CI

Parades aux risques : si MQTT dérape, livrer d'abord la voie REST (même modèle de données) et brancher MQTT en simple adaptateur ; l'idempotence est testée avant tout branchement de source réelle.

3.10 Sprint 7 — Fonctions avancées

Objectif	Différenciation par rapport aux solutions existantes.
Période	06/10/2026 → 19/10/2026 (14 jours) — 57 pts / capacité 38 (surcharge, cf. §3.3).
Démo	Démonstration carte, météo, QR code, détection d'anomalie.
Risques	Microservice Python, intégration REST.
Burndown idéal	J1 : 57 — J4 : 43 — J7 : 29 — J10 : 14 — J14 : 0.

ID	Story	Pts
US-028	Photos d'inspection	5
US-029	Contexte météo local	5
US-030	Carte et rayons de butinage	8
US-031	Liste de tâches et rappels	5
US-032	Suivi de la reine	5
US-033	Récolte et QR code	8
US-034	Détection d'anomalie adaptative (EWMA)	13
US-035	Interface microservice IA Python	8

3.10.1 Plan d'exécution

Tâche	Stories	j-h	Livrable
Photos d'inspection : réutilisation du module photo de S3 attaché à l'inspection	US-028	1	Photos liées à l'inspection
Carte Leaflet + OpenStreetMap : sites + cercles de butinage 1/2/3 km	US-030	2	Carte interactive
API météo par géolocalisation + cache local (quota d'appels)	US-029	1,5	Contexte météo sur la visite
Liste de tâches et rappels (calendrier)	US-031	1,5	Rappels démontrables
Suivi de la reine : historique de statut	US-032	1	Chronologie reine
Récolte : lot + QR code + page publique de traçabilité	US-033	2	QR scannable de bout en bout
Microservice Python EWMA : ligne de base par ruche + z-score, API REST/JSON	US-034/035	3	Conteneur IA dans docker-compose
Intégration EWMA → alertes sanitaires (remplace le seuil simple de S5)	US-034	1	Alerte adaptative en démo

Parades aux risques : le microservice IA est découplé par contrat REST : s'il n'est pas prêt, l'application reste sur le seuil simple de S5 sans autre impact. US-029/US-033 sont les fusibles du sprint (priorité moyenne, aucune dépendance descendante).

3.11 Sprint 8 — Qualité & livraison

Objectif	Livrer une solution complète et documentée.	
Période	20/10/2026 → 02/11/2026 (14 jours) — 44 pts / capacité 38.	
Démo	Démonstration finale complète + soutenance.	
Risques	Couverture de tests, délais de documentation.	
Burndown idéal	J1 : 44 — J4 : 33 — J7 : 22 — J10 : 11 — J14 : 0.	

ID	Story	Pts
US-023	Chiffrement SSL/TLS X.509	5
US-036	Tests unitaires JUnit 5 + Mockito	8
US-037	Tests d'intégration (Arquillian)	5
US-038	Tests de charge k6	5
US-039	Diagrammes UML complets	13
US-040	Rapport + poster + présentation	8

3.11.1 Plan d'exécution

Tâche	Stories	j-h	Livrable
SSL/TLS : certificats X.509, HTTPS forcé, redirection 80 → 443	US-023	1,5	Chaîne TLS validée
Campagne de tests unitaires : combler les trous jusqu'à 70 % de couverture métier	US-036	3	Rapport JaCoCo $\geq 70\%$
Compléter les tests d'intégration Arquillian (workflow, sync, API)	US-037	2	Suite d'intégration complète
Tests de charge k6 + corrections de performance	US-038	2	p95 < 500 ms, erreurs < 1 %
Finalisation des 10 diagrammes UML (maintenus depuis S2, cf. §3.3.4)	US-039	2,5	Dossier UML complet
Rapport, poster, présentation + répétition de soutenance	US-040	3	Livrables de soutenance
<i>Feature freeze</i> au jour 10 : corrections et empaquetage uniquement	—	—	v1.0.0 taguée et déployée

Parades aux risques : la couverture de tests ne se rattrape pas en un sprint — d'où le quality gate progressif (chapitre 4) qui doit amener la couverture à $\approx 65\%$ dès la fin de S7 ; les diagrammes UML maintenus au fil de l'eau réduisent US-039 à une passe de mise en cohérence.

3.12 Suivi en cours de sprint

Modalités de suivi. Le *burndown* réel est mis à jour quotidiennement au daily scrum. En fin de sprint, la rétrospective consigne : ce qui a bien fonctionné, ce qui peut être amélioré, et les actions pour le sprint suivant. Les fichiers éditables `SPRINT-0X.md` du dossier `zumm_scrum_devops_roadmap/02_sprints/` servent de support opérationnel ; ce document en est la vue consolidée.

CHAPITRE 4

Pipeline DevOps (CI/CD)

La chaîne CI/CD s'appuie sur **GitHub Actions** (ou GitLab CI au choix) et se déclenche à chaque *push* ou *pull request*.

4.1 Environnements

Environnement	Branche	URL	Usage
Local	feature/*	http://localhost:8080/zumm	Développement local (WildFly)
Dev	develop	https://dev.zumm.local	Intégration continue, tests
Staging	release/*	https://staging.zumm.local	Recette, démo sprint review
Production	main	https://zumm.local	Déploiement manuel validé

4.2 Vue d'ensemble du pipeline

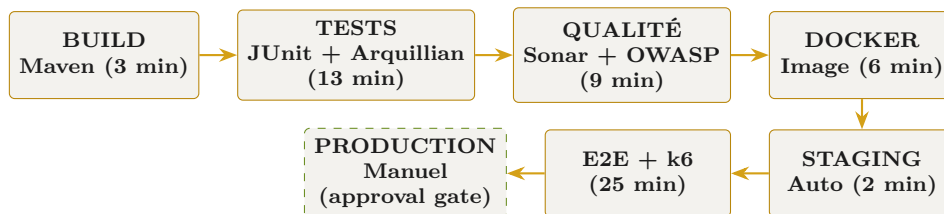


Figure 4.1. Enchaînement des étapes du pipeline CI/CD (durée totale ≈ 1 h).

4.3 Détail des étapes

Étape	Description	Commandes	Seuils / conditions	Durée
Build	Compilation Maven + packaging WAR/EAR (JDK 17, Maven 3.9+, WildFly)	<code>mvn clean compile</code> <code>mvn package -DskipTests</code>	—	3 min
Tests unitaires	JUnit 5 + Mockito + JCoCo	<code>mvn test</code> <code>mvn jacoco:report</code>	couverture $\geq 70\%$, 0 échec	5 min
Tests d'intégration	Arquillian <i>in-container</i> + H2	<code>mvn verify -P integration-tests</code>	—	8 min
Analyse qualité	SonarQube + SpotBugs + Checkstyle	<code>mvn sonar:sonar</code> <code>mvn spotbugs:check</code> <code>mvn checkstyle:check</code>	0 bug, 0 vulnérabilité, < 50 <i>code smells</i>	5 min
Sécurité	Scan des dépendances OWASP	<code>mvn dependency-check:check</code>	—	4 min
Build Docker	Image WildFly + WAR Zümm	<code>docker build -t zumm:\${VERSION} .</code>	—	6 min
Déploiement staging	Déploiement automatique	<code>docker-compose -f docker-compose.staging.yml up -d</code>	branches <code>release/*</code> , <code>main</code>	2 min
Tests E2E	Selenium / Cypress sur staging	<code>mvn test -P e2e</code>	—	10 min
Tests de charge	k6, performance et robustesse	<code>k6 run tests/load/</code>	p95 < 500 ms, erreurs $< 1\%$	15 min
Déploiement production	Déploiement manuel (<i>approval gate</i>)	<code>docker-compose -f docker-compose.prod.yml up -d</code>	branche <code>main</code> + <i>ap-probation</i>	2 min

4.4 Infrastructure as Code

- **Docker** : Dockerfile applicatif (WildFly + WAR), Dockerfile base de données (PostgreSQL + PostGIS), `docker-compose.yml` pour la stack complète (application + PostgreSQL + Nginx + monitoring) ;
- **Kubernetes** (optionnel, scaling futur) : manifestes `deployment.yml`, `service.yml`, `ingress.yml`, `configmap.yml`.

4.5 Mise en place progressive

Le pipeline complet décrit ci-dessus est la cible ; sa mise en place suit la montée en puissance du projet :

- **CI dès le sprint 1** (build + tests + Sonar), mais CD staging seulement à partir du sprint 2 : inutile de maintenir quatre environnements avant d'avoir un WAR stable ;

- **Quality gate progressif** : exiger 70 % de couverture dès le sprint 1 est irréaliste sur du code d'infrastructure — commencer à 50 % et monter de 5 points par sprint jusqu'à la cible ;
- **Gestion des secrets** (GitHub Secrets / vault) intégrée au pipeline avant le sprint 4 : l'authentification OAuth (US-020) en dépend ;
- **Tests E2E et de charge** activés à partir du sprint 5, quand les parcours utilisateur sont stabilisés.

4.6 Stratégie de branches Git

Branche	Rôle
main	Production stable
develop	Intégration continue
release/*	Stabilisation d'une release
feature/*	Développement d'une fonctionnalité
hotfix/*	Correction urgente en production

Fichiers opérationnels. Les fichiers exécutables du pipeline (`github-actions.yml`, `Dockerfile`, `docker-compose.yml`) sont maintenus dans `zumm_scrum_devops_roadmap/03_devops_pipeline/` ; ce chapitre en décrit le contrat (étapes, seuils, conditions).

CHAPITRE 5

Plan de releases

Le versionnage suit **Semantic Versioning** (SemVer : MAJOR.MINOR.PATCH). Quatre releases jalonnent le projet, une à l'issue de chaque paire de sprints.

5.1 Synthèse

Version	Nom	Date prévue	Sprints
v0.1.0	MVP — Fondation	11/08/2026	S1, S2
v0.2.0	Visites & collaboration	08/09/2026	S3, S4
v0.3.0	Analytics & dashboards	06/10/2026	S5, S6
v1.0.0	Release Candidate	02/11/2026	S7, S8

5.2 v0.1.0 — MVP : Fondation

Contenu	CRUD complet des entités métier ; architecture J2EE multi-niveaux ; composition de la ruche (corps/hausse/cadre) ; configuration <code>ConfigZumm.ini</code> ; i18n FR/EN/AR.
Critère d'acceptation	Application déployable avec données de test.
Statut	Planifiée.

5.3 v0.2.0 — Visites & collaboration

Contenu	Workflow planification/approbation de visite ; rapport de visite complet ; mode hors-ligne PWA ; authentification OAuth 2.0 ; sécurité SSL/X.509.
Critère d'acceptation	Parcours utilisateur complet en ligne et hors-ligne.
Statut	Planifiée.

5.4 v0.3.0 — Analytics & dashboards

Contenu	Calendrier agents × ruches ; tableaux de bord production & alertes ; synthèse ROI ; ingestion des mesures capteurs ; service web API (SOAP/XML).
Critère d'acceptation	Dashboards fonctionnels avec données réelles.
Statut	Planifiée.

5.5 v1.0.0 — Release Candidate

Contenu	Fonctionnalités avancées (carte, météo, QR code) ; détection d'anomalie adaptative (IA) ; tests complets (unitaires/intégration/charge) ; documentation & maintenance ; monitoring & alerting.
Critère d'acceptation	Solution complète prête pour la production.
Statut	Planifiée.

Règle de livraison. Une release n'est publiée que si tous les critères de la Definition of Done (§1.2.4) sont satisfaits pour l'ensemble des stories concernées, et si le pipeline CI/CD (chapitre 4) est intégralement vert sur la branche `release/*` correspondante.

CHAPITRE 6

Monitoring & observabilité

L'observabilité repose sur **Prometheus** + **Grafana** (métriques), **ELK** ou **Loki** (logs), **AlertManager** (alertes) et **Jaeger** (tracing distribué).

6.1 Dashboards Grafana

6.1.1 Zümm — Vue d'ensemble (métier)

Widget	Type	Requête PromQL
Ruches actives	stat	<code>count(ruche_status{status='active'})</code>
Visites aujourd'hui	stat	<code>sum(visites_total{date='today'})</code>
Production miel (kg)	graph	<code>sum(production_miel_kg) by (site)</code>
Alertes actives	graph	<code>count(alertes_status{status='active'})</code>
Latence API (p95)	heatmap	<code>histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[5m]))</code>

6.1.2 Zümm — Performance (technique)

Widget	Type	Requête PromQL
CPU JVM	graph	<code>jvm_cpu_usage_percent</code>
Mémoire heap	graph	<code>jvm_memory_heap_used_bytes</code>
Connexions DB	graph	<code>jdbc_connections_active</code>
Requêtes HTTP/s	graph	<code>rate(http_requests_total[1m])</code>

6.1.3 Zümm — Sécurité

Widget	Type	Requête PromQL
Tentatives échouées	stat	<code>sum(auth_failures_total[1h])</code>
Erreurs 4xx/5xx	graph	<code>sum(rate(http_errors_total[5m])) by (status)</code>
Top 10 IP suspectes	table	<code>topk(10, rate(requests_by_ip[5m]))</code>

6.2 Règles d'alerte

Alerte	Condition	Sévérité	Action
Latence API élevée	p95 sur 5 min > 500 ms	Warning	Notifier Slack + escalade si persistant
Taux d'erreurs > 1 %	<code>http_errors</code> / <code>http_requests</code> > 0,01	Critical	PagerDuty + rollback automatique
Connexions DB épuisées	actives / max > 0,9	Critical	Scaler le pool + alerte ops
JVM heap > 85 %	heap utilisé / heap max > 0,85	Warning	Analyse mémoire + redémarrage planifié

6.3 Gestion des logs

- **Niveaux** : ERROR, WARN, INFO, DEBUG ;
- **Corrélation** : en-tête `X-Request-ID` propagé pour le tracing distribué ;
- **Rétention** : 30 jours en stockage chaud, 1 an en stockage froid (S3).

Boucle DevOps. Les métriques métier (production, visites, alertes sanitaires) sont exposées par l'application elle-même : les dashboards de supervision technique et les tableaux de bord fonctionnels du cahier des charges (§4.3) partagent ainsi la même source de vérité.